



# Logic Gates in Python

Last Updated : 11 Jul, 2025

**Logic gates** are the basic building blocks of **digital systems**. They work by taking one or more binary inputs (0 or 1) and giving a single binary output based on a specific rule of logic. Each gate performs a different function depending on the values it receives. Using Python, we can easily simulate the behavior of these gates through simple code. This makes it easier to understand how digital circuits work and how decisions are made in computing, without needing any physical hardware.

## Types of Logic Gates in Python

There are seven basic logic gates in [Python](#). These are the following:

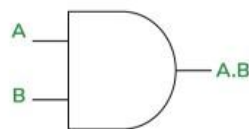
### Table of Content

- [AND Gate](#)
- [OR Gate](#)
- [NOT Gate](#)
- [NAND Gate](#)
- [NOR Gate](#)
- [XOR Gate](#)
- [XNOR](#)

## AND Gate

[AND gate](#) checks if both things are true. It only gives 1 (true) when both inputs are 1. If even one input is 0, it gives 0.

### 2- Input AND Gate



Truth Table

| A (Input 1) | B (Input 2) | X = (A.B) |
|-------------|-------------|-----------|
| 0           | 0           | 0         |
| 0           | 1           | 0         |
| 1           | 0           | 0         |
| 1           | 1           | 1         |

AND Gate

### Example:

```
print("A B | Output")
for a in [0, 1]:
    for b in [0, 1]:
        res = a & b
        print(f"{a} {b} | {res}")
```

Loading Playground...



## Output

| A | B | Output |
|---|---|--------|
| 0 | 0 | 0      |
| 0 | 1 | 0      |
| 1 | 0 | 0      |
| 1 | 1 | 1      |

**Explanation:** **Outer loop** goes through the values of **a**, while the **inner loop** goes through the values of **b**. For each pair, it calculates the bitwise AND (**a & b**), which results in 1 only if both **a** and **b** are 1, otherwise the result is 0.

## OR Gate

The [OR gate](#) checks if at least one thing is true. If any input is 1, it gives 1. It gives 0 only when both inputs are 0.

2-Input OR Gate



Truth Table

| Input A | Input B | Output |
|---------|---------|--------|
| 0       | 0       | 0      |
| 0       | 1       | 1      |
| 1       | 0       | 1      |
| 1       | 1       | 1      |

OR Gates

## Example:

```
print("A B | Output")
for a in [0, 1]:
    for b in [0, 1]:
        res = a | b
        print(f"{a} {b} | {res}")
```

Loading Playground...



## Output

| A | B | Output |
|---|---|--------|
| 0 | 0 | 0      |
| 0 | 1 | 1      |
| 1 | 0 | 1      |
| 1 | 1 | 1      |

**Explanation:** **Outer loop** iterates through the values of **a**, while the **inner loop** iterates through the values of **b**. For each pair, it calculates the bitwise OR (**a | b**), which results in 1 if either **a** or **b** is 1, otherwise the result is 0.

## NOT Gate

[NOT gate](#) is different from the others, it takes just one input and flips it. If the input is 1, it becomes 0. If it's 0, it becomes 1. It's also called an inverter.

## NOT Gate



Truth Table

| A (Input) | Y = $\bar{A}$ (Output) |
|-----------|------------------------|
| 0         | 1                      |
| 1         | 0                      |

NOT Gate

## Example:

```
print("A | Output")
for a in [0, 1]:
    res = 1 if a == 0 else 0
    print(f"{a} | {res}")
```



## Output

```
A | Output
0 | 1
1 | 0
```

**Explanation:** Loop iterates through the values of **a** (0 and 1). For each value of **a**, it checks if **a** is 0. If **a** is 0, the **res** is set to 1, otherwise it is set to 0.

## NAND Gate

NAND gate is the opposite of the AND gate. It gives 0 only when both inputs are 1. In all other cases, it gives 1.

## 2-Input NAND Gate



Truth Table

| Input A | Input B | X = (A.B)' |
|---------|---------|------------|
| 0       | 0       | 1          |
| 0       | 1       | 1          |
| 1       | 0       | 1          |
| 1       | 1       | 0          |

NAND Gate

## Example:

```
print("A B | Output")
for a in [0, 1]:
    for b in [0, 1]:
        res = 0 if a & b else 1
```

```
print(f"{a} {b} | {res}")
```

× ▶ 📄

## Output

```
A B | Output
0 0 | 1
0 1 | 1
1 0 | 1
1 1 | 0
```

**Explanation:** **Outer loop** iterates through the values of **a**, while the **inner loop** iterates through the values of **b**. For each pair of **a** and **b**, it performs the bitwise AND operation (**a & b**). If the result of the AND operation is 1, the output (**res**) is set to 0 (since NAND is the opposite of AND). Otherwise, the result is set to 1.

## NOR Gate

[NOR gate](#) is the opposite of the OR gate. It gives 1 only when both inputs are 0. If even one input is 1, it gives 0.

2- Input NOR Gate



Truth Table

| Input A | Input B | $0 = (A + B)'$ |
|---------|---------|----------------|
| 0       | 0       | 1              |
| 0       | 1       | 0              |
| 1       | 0       | 0              |
| 1       | 1       | 0              |

NOR Gate

## Example:

```
print("A B | Output")
for a in [0, 1]:
    for b in [0, 1]:
        res = 0 if a | b else 1
        print(f"{a} {b} | {res}")
```

× ▶ 📄

## Output

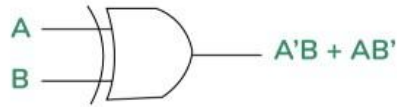
```
A B | Output
0 0 | 1
0 1 | 0
1 0 | 0
1 1 | 0
```

**Explanation:** **Outer loop** iterates over **a** and the inner loop over **b**. For each pair, it performs a bitwise OR (**a | b**). If the result is 1 (either **a** or **b** is 1), **res** is set to 0. If both are 0, **res** becomes 1, simulating a NOR gate.

## XOR Gate

XOR gate is a little smart, it gives 1 when the inputs are different. If both inputs are the same, it gives 0.

### XOR Gate



Truth Table

| A (Input 1) | B (Input 2) | X = A'B + AB' |
|-------------|-------------|---------------|
| 0           | 0           | 0             |
| 0           | 1           | 1             |
| 1           | 0           | 1             |
| 1           | 1           | 0             |

XOR Gate

### Example:

```
print("A B | Output")
```

a Types Interview Questions Examples Quizzes DSA Python Data Science NumPy Pandas Practice Djan



Sign In

```
for a in [0, 1]:
    res = a ^ b
    print(f"{a} {b} | {res}")
```



### Output

```
A B | Output
0 0 | 0
0 1 | 1
1 0 | 1
1 1 | 0
```

**Explanation:** Outer loop goes through values of **a** and the inner loop through **b**. For each pair, it performs a bitwise XOR ( $a \wedge b$ ). The result is 1 only if **a** and **b** are different otherwise, it's 0.

### XNOR

XNOR gate is the opposite of XOR. It gives 1 when the inputs are the same. If the inputs are different, it gives 0.

### What is XNOR Gate?



Truth Table

| Input A | Input B | Output |
|---------|---------|--------|
| 0       | 0       | 1      |
| 0       | 1       | 0      |
| 1       | 0       | 0      |
| 1       | 1       | 1      |

XNOR Gate

```
print("A B | Output")
for a in [0, 1]:
    for b in [0, 1]:
        res = 1 if a == b else 0
```



```
print(f"{a} {b} | {res}")
```

## Output

| A | B | Output |
|---|---|--------|
| 0 | 0 | 1      |
| 0 | 1 | 0      |
| 1 | 0 | 0      |
| 1 | 1 | 1      |

**Explanation:** Outer loop iterates over values of **a** and the inner loop over **b**. For each pair, it checks if **a** and **b** are equal. If they are the same, **res** is set to 1 otherwise, it's 0.

[Comment](#)

A aishwa...

28

Article Tags: [Python](#)

## Explore

[Python Fundamentals](#)
[Python Data Structures](#)
[Advanced Python](#)
[Data Science with Python](#)
[Web Development with Python](#)
[Python Practice](#)


**Corporate & Communications Address:**  
A-143, 7th Floor, Sovereign Corporate Tower, Sector- 136, Noida, Uttar Pradesh (201305)

**Registered Address:**  
K 061, Tower K, Gulshan Vivante Apartment, Sector 137, Noida, Gautam Buddh Nagar, Uttar Pradesh, 201305

### Company

[About Us](#)  
[Legal](#)  
[Privacy](#)  
[Policy](#)  
[Contact Us](#)  
[Advertise with us](#)  
[GFG Corporate Solution](#)  
[Training Program](#)

### Explore

[POTD](#)  
[Job-A-Thon](#)  
[Blogs](#)  
[Nation](#)  
[Skill Up](#)

### Tutorials

[Programming Languages](#)  
[DSA](#)  
[Web](#)  
[Technology](#)  
[AI, ML & Data Science](#)  
[DevOps](#)  
[CS Core](#)  
[Subjects](#)  
[Interview Preparation](#)  
[Software and Tools](#)

### Courses

[ML and Data Science](#)  
[DSA and Placements](#)  
[Web Development](#)  
[Programming Languages](#)  
[DevOps & Cloud](#)  
[GATE](#)  
[Trending Technologies](#)

### Videos

[DSA](#)  
[Python](#)  
[Java](#)  
[C++](#)  
[Web Development](#)  
[Data Science](#)  
[CS Subjects](#)

### Preparation Corner

[Interview Corner](#)  
[Aptitude](#)  
[Puzzles](#)  
[GfG 160](#)  
[System Design](#)



@GeeksforGeeks, Sanchhaya Education Private Limited, All rights reserved